Introduction & Dataset Description:

The dataset used in this project is the FC 26 (EA Sports FC 2026) Player Dataset, containing detailed player attributes, performance ratings, financial information, and demographic data about professional football players from leagues around the world. The dataset is widely used in sports analytics research for player evaluation, performance prediction, and exploratory data analysis. The dataset provides a substantial number of player records, making it a reliable foundation for performing data cleaning, exploration data analysis, and machine learning-based prediction tasks.

Dataset Reference: Kaggle. (2025). *FC 26 Player Dataset*. Kaggle. https://www.kaggle.com/datasets/rovnez/fc-26-fifa-26-player-data/data

Selected Attributes:

1. **short_name (Player Name):** A character variable representing the short display name of the player.
2. **player_positions:** A categorical variable indicating the playing position(s) of the player on the field (e.g., ST, CB, GK, CM).
3. **overall:** Continuous numerical variable representing the player's current overall performance rating on a scale of 0–99.
4. **potential:** Continuous numerical variable representing the player's maximum projected rating, reflecting future growth capacity.
5. **value_eur (Market Value in EUR):** Continuous variable showing the estimated transfer market value of the player in Euros.
6. **wage_eur (Weekly Wage in EUR):** Continuous variable indicating the player's weekly wage in Euros; higher wages typically reflect better-performing players.
7. **age:** Continuous numerical variable representing the player's age in years.
8. **height_cm (Height in cm):** Continuous variable measuring the player's height in centimeters; relevant for position-specific physical analysis.
9. **weight_kg (Weight in kg):** Continuous variable measuring the player's body weight in kilograms; important for physical profiling.
10. **preferred_foot:** Binary categorical variable indicating the player's dominant foot.
    - Right = Right-footed
    - Left = Left-footed
11. **pace:** Continuous variable reflecting the player's speed attributes; commonly used in evaluating attackers and wing players.

12. **shooting:** Continuous variable measuring the player's shooting ability; commonly used in forward player analysis.
13. **passing:** Continuous variable representing the player's passing accuracy and vision; key for midfield evaluation.
14. **dribbling:** Continuous variable measuring the player's ball control and dribbling skill; typically elevated in attacking players.
15. **defending:** Continuous variable indicating the player's defensive capability; high values are typical for center-backs and defensive midfielders.
16. **physic:** Continuous variable showing total physical attributes including strength and stamina, reflecting the player's physical presence on the field.

Data Import and View

This step loads the CSV file into R, ensuring proper column headers and delimiter settings, then opens it for inspection in RStudio's data viewer.

Code:

```
df <- read.csv("C:\\Users\\towhi\\Downloads\\archive (3)\\FC26_20250921.csv")

df <- df[, c("short_name", "player_positions", "overall", "potential",
             "value_eur", "wage_eur", "age", "height_cm", "weight_kg",
             "preferred_foot", "pace", "shooting", "passing",
             "dribbling", "defending", "physic")]
view(df)
```

Output:
The output shows the dataset loaded successfully.

Filter

	short_name	player_positions	overall	potential	value_eur	wage_eur
1	J. Bellingham	CAM, CM	90	94	174500000	320000
2	F. Valverde	CM, CDM, RB	89	90	120500000	340000
3	J. Kimmich	CDM, RB, CM	89	89	86000000	140000
4	A. Hakimi	RB, RM	89	90	111000000	170000
5	N. Barella	CM	87	87	79500000	69000
6	H. Çalhanoğlu	CDM, CM	86	86	47500000	61000
7	F. Dimarco	LB, LM	85	85	52000000	54000
8	A. Mac Allister	CM, CDM	87	88	93000000	180000
9	Rodri	CDM, CM	90	90	102000000	270000
10	A. Griezmann	ST, LM, CAM	85	85	26000000	115000
11	T. Hernández	LB, LM	84	85	44500000	65000
12	Bruno Fernandes	CAM, CM	87	87	77000000	165000
13	J. Alvarez	ST	87	90	107000000	125000
14	T. Alexander-Arnold	RB, RM	86	87	73000000	250000
15	Sancet	CAM, CM, ST	84	88	58500000	66000

Showing 1 to 15 of 18,405 entries, 16 total columns

Dataset Dimensions:

Code:

```
cat("Number of rows:", nrow(data), "\n")
cat("Number of columns:", ncol(data))
```

Output:

```
> cat("Number of rows:", nrow(df), "\n")
Number of rows: 18405
> cat("Number of columns:", ncol(df))
Number of columns: 16
>
```

The output shows that the dataset contains 18,405 player records and 110 columns representing various player attributes. Since 110 columns is too many for focused analysis, we will select only 16 key variables relevant to our project.

Dataset Structure:

Code:

```
str(df)
```

Output:
The str() function provides the internal structure of the dataframe. The output lists each variable name, its data type (character, numeric, factor), and example values.

```
> str(df)
'data.frame':   18405 obs. of  16 variables:
 $ short_name      : chr  "J. Bellingham" "F. Valverde" "J. Kimmich" "A. Hakimi"
...
 $ player_positions: chr  "CAM, CM" "CM, CDM, RB" "CDM, RB, CM" "RB, RM" ...
 $ overall         : int  90 89 89 89 87 86 85 87 90 85 ...
 $ potential       : int  94 90 89 90 87 86 85 88 90 85 ...
 $ value_eur       : int  174500000 120500000 86000000 111000000 79500000 47500000
52000000 93000000 102000000 26000000 ...
 $ wage_eur        : int  320000 340000 140000 170000 69000 61000 54000 180000 270
000 115000 ...
 $ age             : int  22 26 30 26 28 31 27 26 29 34 ...
 $ height_cm       : int  186 182 177 181 175 178 175 176 190 176 ...
 $ weight_kg       : int  75 74 75 73 68 69 76 72 82 67 ...
 $ preferred_foot  : chr  "Right" "Right" "Right" "Right" ...
 $ pace            : int  80 88 72 92 80 71 80 66 65 77 ...
 $ shooting        : int  86 84 74 79 78 81 78 82 80 86 ...
 $ passing         : int  83 84 89 82 84 87 84 85 86 85 ...
 $ dribbling       : int  90 84 84 83 86 82 82 85 84 88 ...
 $ defending       : int  78 83 83 82 81 81 79 78 86 59 ...
 $ physic          : int  85 85 79 79 76 73 77 76 85 73 ...
>
```

Introducing Intentional Data Issues:

The following code intentionally introduces missing values, noisy/invalid values, and extreme outliers into the dataset to simulate real world data challenges.

1. **Missing Values:**

Code:

```
df$age[100] <- NA
df$height_cm[300] <- NA
df$weight_kg[400] <- NA
df$overall[500] <- NA
df$potential[800] <- NA
```

Output:

```
> df$age[100]
[1] 30
> df$age[100] <- NA
> df$age[100]
[1] NA
>
> df$height_cm[300]
[1] 175
> df$height_cm[300] <- NA
> df$height_cm[300]
[1] NA
>
> df$weight_kg[400]
[1] 79
> df$weight_kg[400] <- NA
> df$weight_kg[400]
[1] NA
>
> df$overall[500]
[1] 74
> df$overall[500] <- NA
> df$overall[500]
[1] NA
>
> df$potential[800]
[1] 79
> df$potential[800] <- NA
> df$potential[800]
[1] NA
```

The output confirms that each value was successfully replaced with NA.

2. **Noisy or Invalid Values**

Code:

```
df$age[350] <- 999
df$value_eur[8000] <- -99999
df$physic[1100] <- -10
df$preferred_foot[200] <- "Hand"
df$preferred_foot[3000] <- "none"
df$height_cm[450] <- 999
df$shooting[5500] <- -50
df$weight_kg[12000] <- -30
df$passing[7777] <- 999
df$age[16000] <- -12
```

Output:

```
> df$age[350]
[1] 31
> df$age[350] <- 999
> df$age[350]
[1] 999
>
> df$value_eur[8000]
[1] 1800000
> df$value_eur[8000] <- -99999
> df$value_eur[8000]
[1] -99999
>
> df$physic[1100]
[1] 80
> df$physic[1100] <- -10
> df$physic[1100]
[1] -10
>
> df$preferred_foot[200]
[1] "Right"
> df$preferred_foot[200] <- "Hand"
> df$preferred_foot[200]
[1] "Hand"
>
> df$preferred_foot[3000]
[1] "Right"
> df$preferred_foot[3000] <- "none"
> df$preferred_foot[3000]
[1] "none"
```

```
> df$height_cm[450]
[1] 191
> df$height_cm[450] <- 999
> df$height_cm[450]
[1] 999
>
> df$shooting[5500]
[1] 54
> df$shooting[5500] <- -50
> df$shooting[5500]
[1] -50
>
> df$weight_kg[12000]
[1] 69
> df$weight_kg[12000] <- -30
> df$weight_kg[12000]
[1] -30
>
> df$passing[7777]
[1] 58
> df$passing[7777] <- 999
> df$passing[7777]
[1] 999
>
> df$age[16000]
[1] 32
> df$age[16000] <- -12
> df$age[16000]
[1] -12
```

The output displays each row's original value before modification and the new invalid value after replacement. For example, Row 350's age was changed from its original realistic value to 999, which is humanly impossible.

3. **Outliers Inserted**

Code:

```
df$wage_eur[900] <- 9999999
df$value_eur[4500] <- 750000000
df$wage_eur[11000] <- 7500000
df$weight_kg[6600] <- 350
df$potential[2200] <- 250
df$height_cm[14500] <- 15
df$value_eur[17000] <- 650000000
```

Output:

```
> df$wage_eur[900]
[1] 31000
> df$wage_eur[900] <- 9999999
> df$wage_eur[900]
[1] 9999999
>
> df$value_eur[4500]
[1] 2100000
> df$value_eur[4500] <- 750000000
> df$value_eur[4500]
[1] 7.5e+08
>
> df$wage_eur[11000]
[1] 3000
> df$wage_eur[11000] <- 7500000
> df$wage_eur[11000]
[1] 7500000
>
> df$weight_kg[6600]
[1] 77
> df$weight_kg[6600] <- 350
> df$weight_kg[6600]
[1] 350
>
> df$potential[2200]
[1] 77
> df$potential[2200] <- 250
> df$potential[2200]
[1] 250
```

```
> df$height_cm[14500]
[1] 196
> df$height_cm[14500] <- 15
> df$height_cm[14500]
[1] 15
>
> df$value_eur[17000]
[1] 525000
> df$value_eur[17000] <- 650000000
> df$value_eur[17000]
[1] 6.5e+08
```

The output displays each row's original value before and after inserting the outlier. For example, Row 900's weekly wage was inflated to 9,999,999 EUR, which is far beyond any realistic player salary.

4. **Duplicate Rows Inserted**

Code:

```
cat("Rows before duplicates:", nrow(df), "\n")
df <- rbind(df, df[10, ])
df <- rbind(df, df[25, ])
df <- rbind(df, df[50, ])
df <- rbind(df, df[130, ])
cat("Rows after duplicates:", nrow(df), "\n")
```

Output:

```
> cat("Rows before duplicates:", nrow(df), "\n")
Rows before duplicates: 18405
> df <- rbind(df, df[10, ])
> df <- rbind(df, df[25, ])
> df <- rbind(df, df[50, ])
> df <- rbind(df, df[130, ])
> cat("Rows after duplicates:", nrow(df), "\n")
Rows after duplicates: 18409
```

The output shows the row count before and after inserting duplicates. Four exact copies of existing rows (10, 25, 50, and 130) were appended to the end of the dataset, increasing the total row count by 4.

Handling Missing Values:

1. **Identifying Missing Values**

This code uses colSums(is.na()) to count missing values in each column. It helps identify which variables contain NA entries that require cleaning.

Code:

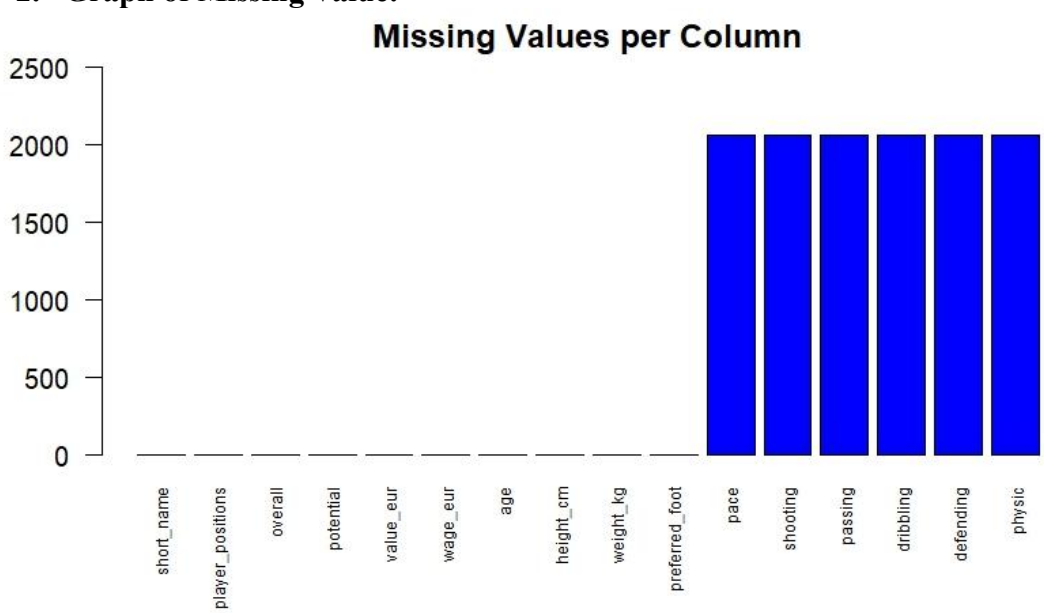
colSums(is.na(df))

Output:

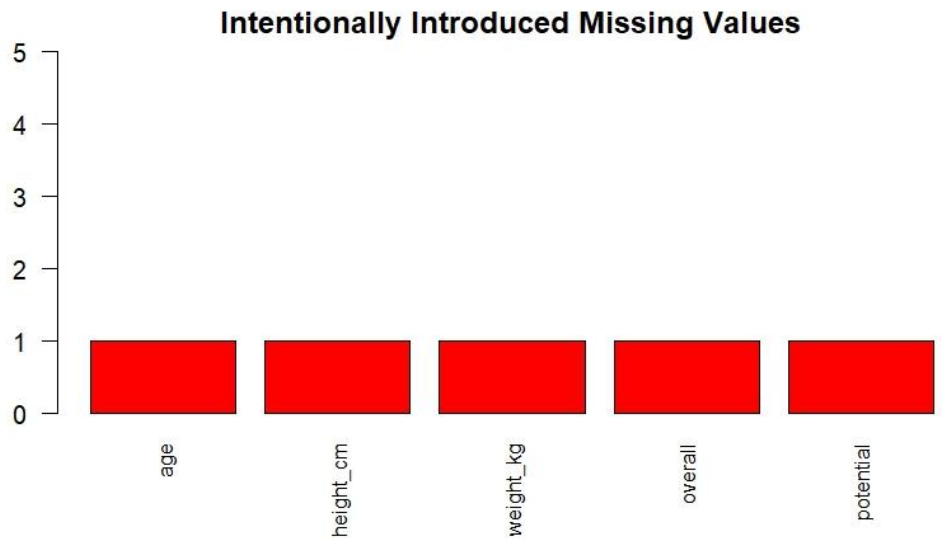
The output shows a numeric count of missing values for each column. Five missing values were intentionally introduced in the age, height_cm, weight_kg, overall, and potential columns (1 each). Additionally, approximately 2062 natural missing values exist in the pace, shooting, passing, dribbling, defending, and physic columns these belong to goalkeeper players who do not have outfield attribute ratings in the FIFA dataset.

```
> colSums(is.na(df))
      short_name player_positions          overall        potential
               0               0                1                1
       value_eur         wage_eur              age        height_cm
               0               0                1                1
       weight_kg   preferred_foot             pace         shooting
               1               0             2062             2062
         passing        dribbling        defending           physic
            2062            2062             2062             2062
> |
```

2. **Graph of Missing Value:**



All columns



Only intentional missing values

3. **Replacing Missing Values**

Code:

With Mean:

```
df$age[is.na(df$age)] <- mean(df$age, na.rm = TRUE)
df$height_cm[is.na(df$height_cm)] <- mean(df$height_cm, na.rm = TRUE)
df$weight_kg[is.na(df$weight_kg)] <- mean(df$weight_kg, na.rm = TRUE)
df$overall[is.na(df$overall)] <- mean(df$overall, na.rm = TRUE)
df$potential[is.na(df$potential)] <- mean(df$potential, na.rm = TRUE)
```

With Median:

```
df$pace[is.na(df$pace)] <- median(df$pace, na.rm = TRUE)
df$shooting[is.na(df$shooting)] <- median(df$shooting, na.rm = TRUE)
df$passing[is.na(df$passing)] <- median(df$passing, na.rm = TRUE)
df$dribbling[is.na(df$dribbling)] <- median(df$dribbling, na.rm = TRUE)
df$defending[is.na(df$defending)] <- median(df$defending, na.rm = TRUE)
df$physic[is.na(df$physic)] <- median(df$physic, na.rm = TRUE)
```

Verify:

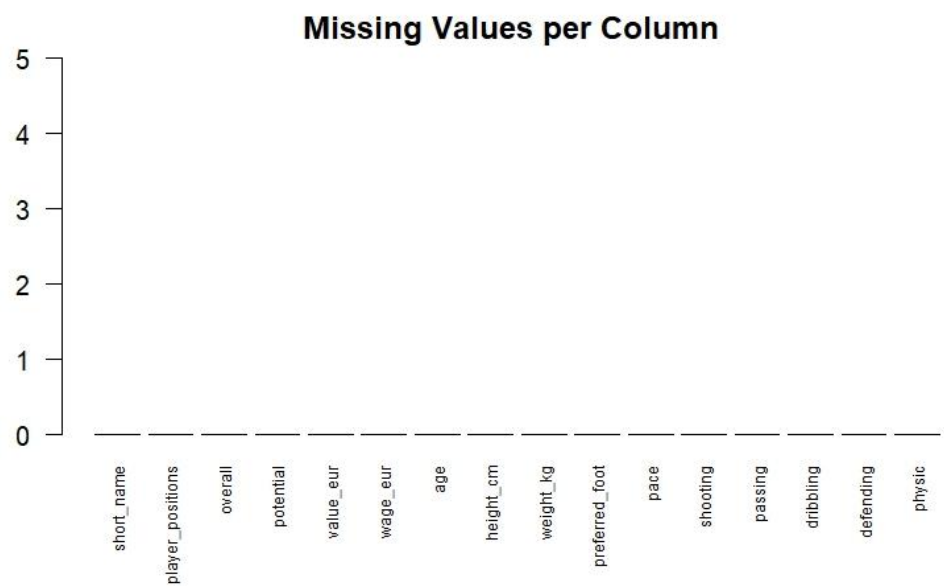
```
colSums(is.na(df))
```

Output:

```
> colSums(is.na(df))
      short_name player_positions          overall
               0                0                0
       potential        value_eur         wage_eur
               0                0                0
             age        height_cm        weight_kg
               0                0                0
  preferred_foot             pace         shooting
               0                0                0
         passing        dribbling        defending
               0                0                0
          physic
               0
>
```

Mean imputation was applied to the 5 intentionally introduced missing values (age, height_cm, weight_kg, overall, potential) as these columns have approximately normal distributions where the mean provides a reliable central estimate. Median imputation was used for the natural goalkeeper missing values (pace, shooting, passing, dribbling, defending, physic) because these columns may contain skewed distributions, and the median is more robust against skewness since it is not influenced by extreme values.

Graph After Removing Missing Values:



Verifying the changed values:

df$age[100]
df$height_cm[300]
df$weight_kg[400]
df$overall[500]
df$potential[800]

Output:

```
> df$age[100]
[1] 25.27336
> df$height_cm[300]
[1] 182.0351
> df$weight_kg[400]
[1] 75.1508
> df$overall[500]
[1] 65.77021
> df$potential[800]
[1] 71.17693
```

The output displays the imputed values for the 5 intentionally missing rows. Row 100's age was replaced with the mean value 25.27, Row 300's height was replaced with 182.04 cm, Row 400's weight with 75.15 kg, Row 500's overall rating with 65.77, and Row 800's potential with 71.18. These values represent the column averages, confirming that mean imputation was applied correctly.

Handling Noisy Values:

Noisy values refer to incorrect or unrealistic data points, like impossible numbers or incorrect categories. Noisy values were adjusted by being substituted with a more plausible value. In the case of numeric variables, the median and mode were chosen as it is not influenced by extreme values. The most frequent category (mode) was used in the case of categorical variables.

Code:

```
df$age[350] <- median(df$age, na.rm = TRUE)
df$age[16000] <- mean(df$age, na.rm = TRUE)
df$value_eur[8000] <- mean(df$value_eur, na.rm = TRUE)
df$physic[1100] <- median(df$physic, na.rm = TRUE)
df$preferred_foot[200] <- "Right"
df$preferred_foot[3000] <- "Left"
df$height_cm[450] <- mean(df$height_cm, na.rm = TRUE)
df$shooting[5500] <- median(df$shooting, na.rm = TRUE)
df$weight_kg[12000] <- mean(df$weight_kg, na.rm = TRUE)
df$passing[7777] <- median(df$passing, na.rm = TRUE)
```

Output:

```
> df$age[350]
[1] 999
> df$age[350] <- median(df$age, na.rm = TRUE)
> df$age[350]
[1] 25
> df$age[16000]
[1] -12
> df$age[16000] <- mean(df$age, na.rm = TRUE)
> df$age[16000]
[1] 25.22045
>
> df$value_eur[8000]
[1] -99999
> df$value_eur[8000] <- mean(df$value_eur, na.rm = TRUE)
> df$value_eur[8000]
[1] 3013292
>
> df$physic[1100]
[1] -10
> df$physic[1100] <- median(df$physic, na.rm = TRUE)
> df$physic[1100]
[1] 66
```

```
> df$preferred_foot[200]
[1] "Hand"
> df$preferred_foot[200] <- "Right"
> df$preferred_foot[200]
[1] "Right"
>
> df$preferred_foot[3000]
[1] "none"
> df$preferred_foot[3000] <- "Left"
> df$preferred_foot[3000]
[1] "Left"
>
>
> df$height_cm[450]
[1] 999
> df$height_cm[450] <- mean(df$height_cm, na.rm = TRUE)
> df$height_cm[450]
[1] 182.0351
>
> df$shooting[5500]
[1] -50
> df$shooting[5500] <- median(df$shooting, na.rm = TRUE)
> df$shooting[5500]
[1] 55
```

```
>
> df$weight_kg[12000]
[1] -30
> df$weight_kg[12000] <- mean(df$weight_kg, na.rm = TRUE)
> df$weight_kg[12000]
[1] 75.1508
>
>
> df$passing[7777]
[1] 999
> df$passing[7777] <- median(df$passing, na.rm = TRUE)
> df$passing[7777]
[1] 58
>
```

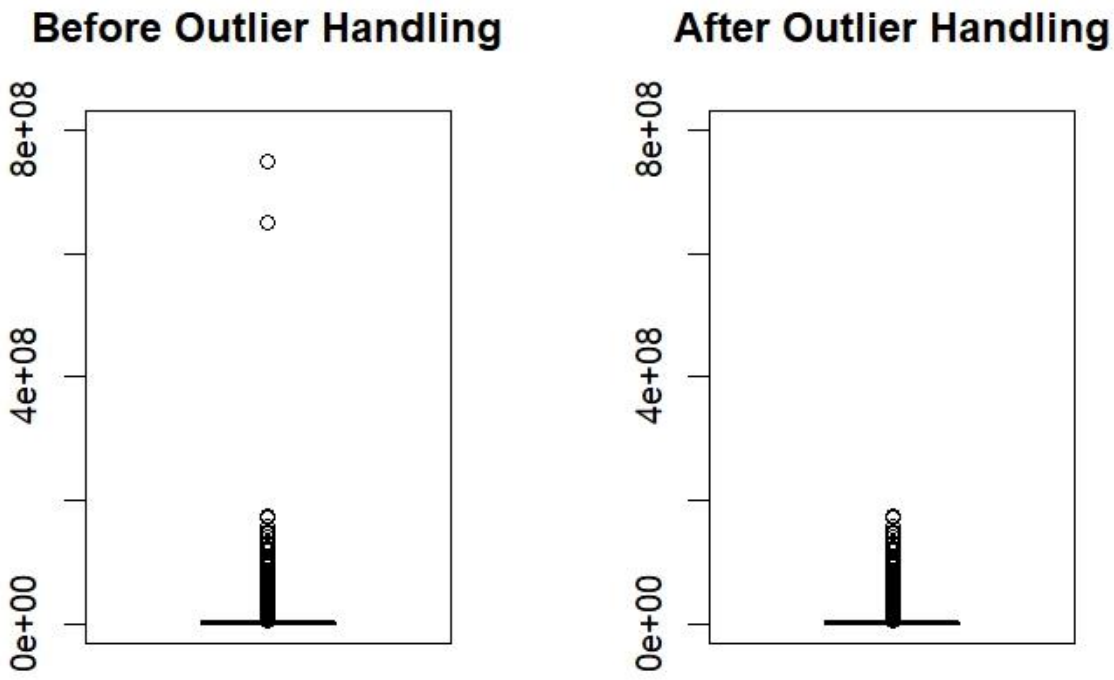
The output confirms all 10 noisy values were successfully corrected. Invalid numeric values like 999, -12, -99999, -10, -50, -30, and 999 were replaced with either the mean or median of their respective columns. For categorical entries, "Hand" was corrected to "Right" and "none" was corrected to "Left". Three methods were used: mean for symmetric columns, median for skewed columns, and mode replacement for categorical columns.

Handling Outliers:

For wage
Code:

```
boxplot(df$wage_eur, main = "wage_eur - Before Outlier Handling")
df$wage_eur[900]
df$wage_eur[900] <- median(df$wage_eur, na.rm = TRUE)
df$wage_eur[900]
df$wage_eur[11000]
df$wage_eur[11000] <- median(df$wage_eur, na.rm = TRUE)
df$wage_eur[11000]
boxplot(df$wage_eur, main = "wage_eur - After Outlier Handling")
```

Output:



The boxplots show value_eur before and after outlier handling. The "Before" plot shows the y-axis stretching up to 600 million EUR due to the two intentionally inserted extreme outliers (750,000,000 and 650,000,000 at rows 4500 and 17000). After replacing these with the median value, the "After" plot shows the y-axis reduced to approximately 150 million EUR.

For Weight:

Code:

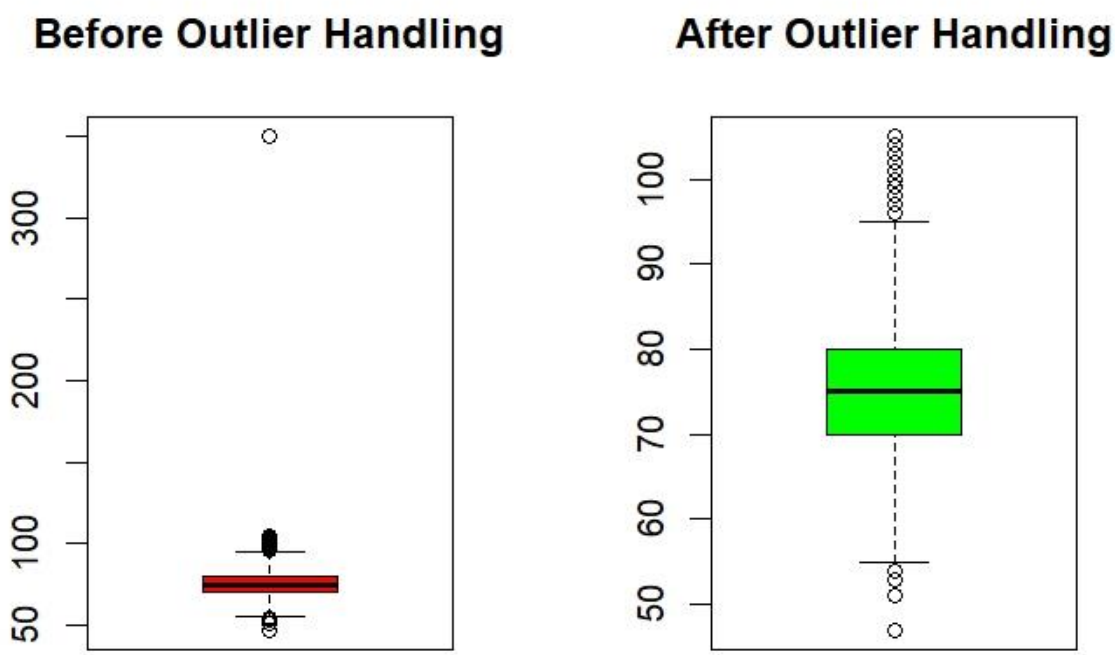
```
par(mfrow = c(1, 2))

df$weight_kg[6600] <- 350
boxplot(df$weight_kg, main = "Before Outlier Handling", col = "red")

df$weight_kg[6600] <- median(df$weight_kg, na.rm = TRUE)
boxplot(df$weight_kg, main = "After Outlier Handling", col = "green")

par(mfrow = c(1, 1))
```

Output:



The boxplots show weight_kg before and after outlier handling. The "Before" plot shows the y-axis stretching up to 350 kg due to the intentionally inserted extreme outlier at row 6600. The box is compressed at the bottom, making it difficult to see the actual data distribution. After replacing the outlier with the median value, the "After" plot clearly displays the true distribution of player weights, ranging from approximately 50 to 100 kg with the median around 75 kg.

Potential outlier:

Code:

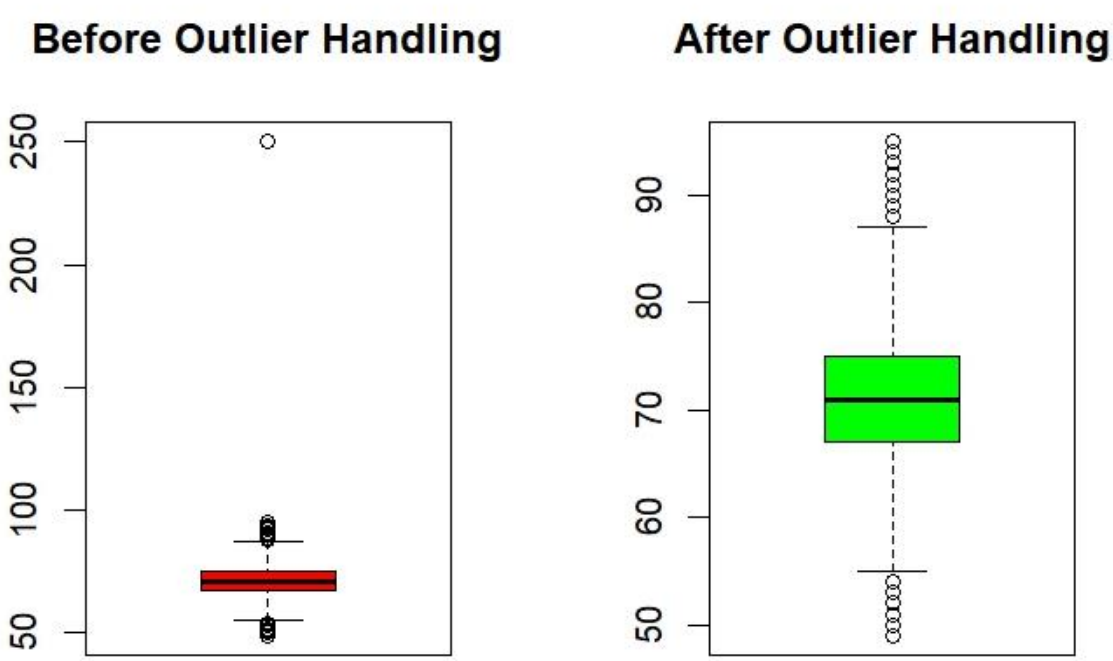
```
dev.off()
par(mfrow = c(1, 2))

df$potential[2200] <- 250
boxplot(df$potential, main = "Before Outlier Handling", col = "red")

df$potential[2200] <- median(df$potential, na.rm = TRUE)
boxplot(df$potential, main = "After Outlier Handling", col = "green")

par(mfrow = c(1, 1))
```

Output:



The boxplots show potential before and after outlier handling. The "Before" plot shows the y-axis reaching 250 due to the intentionally inserted outlier at row 2200, which exceeds the maximum possible rating of 99. The box is compressed at the bottom making the distribution unreadable. After replacing the outlier with the median value, the "After" plot reveals the true distribution of player potential ratings, ranging from approximately 50 to 95 with the median around 71.

height_cm outlier:

Code:

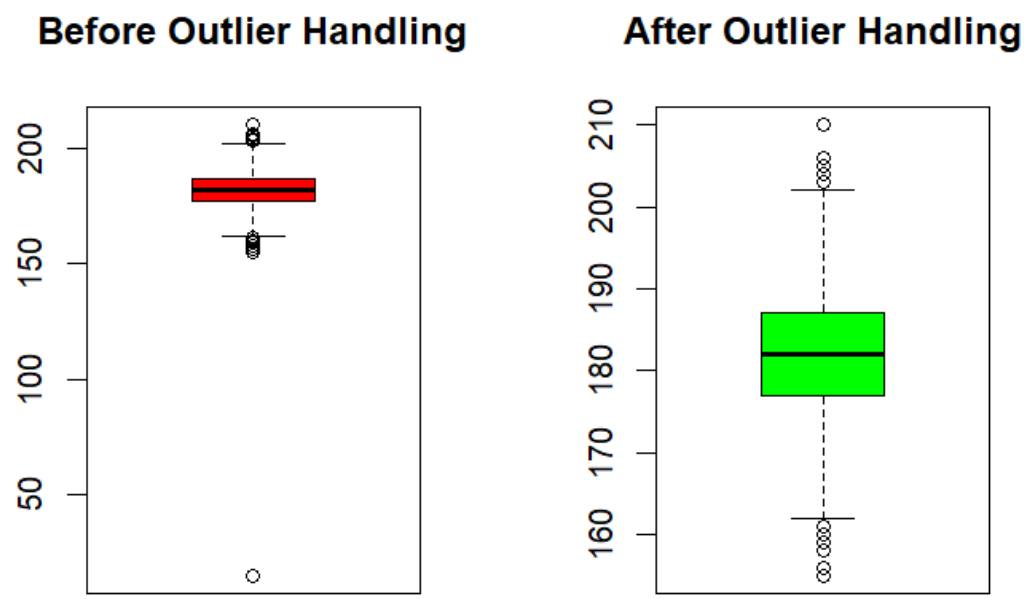
```
dev.off()
par(mfrow = c(1, 2))

df$height_cm[14500] <- 15
boxplot(df$height_cm, main = "Before Outlier Handling", col = "red")

df$height_cm[14500] <- median(df$height_cm, na.rm = TRUE)
boxplot(df$height_cm, main = "After Outlier Handling", col = "green")

par(mfrow = c(1, 1))
```

Output:



The boxplots show height_cm before and after outlier handling. The "Before" plot shows a single point at the bottom near 15 cm, which is the intentionally inserted unrealistic height at row 14500. This extreme low value stretches the y-axis down to nearly 0, compressing the actual data distribution. After replacing the outlier with the median value, the "After" plot clearly shows the true distribution of player heights, ranging from approximately 160 to 210 cm with the median around 182 cm, which reflects realistic heights for professional football players.

Handling Duplicates:

Code:

```
cat("Rows before removing duplicates:", nrow(df), "\n")
sum(duplicated(df))

df <- df[!duplicated(df), ]
cat("Rows after removing duplicates:", nrow(df), "\n")
sum(duplicated(df))
```

Output:

```
> cat("Rows before removing duplicates:", nrow(df), "\n")
Rows before removing duplicates: 18409
> sum(duplicated(df))
[1] 4
>
> df <- df[!duplicated(df), ]
> cat("Rows after removing duplicates:", nrow(df), "\n")
Rows after removing duplicates: 18405
> sum(duplicated(df))
[1] 0
>
```

The duplicated() function was used to detect exact duplicate rows in the dataset. Four duplicate rows (copies of rows 10, 25, 50, and 130) were previously inserted intentionally. The sum(duplicated()) function confirmed the presence of duplicate entries. After applying the removal using df[!duplicated(df),], all duplicate rows were successfully removed. The row count decreased from 18,409 to 18,405, confirming that all 4 intentional duplicates were detected and eliminated.

Normalizing the Dataset:

A min-max normalization function scales selected columns to a 0–1 range: (x - min) / (max - min). The normalized dataset is stored separately to preserve the original.

Code:

```
normalize_dataset <- df
normalize <- function(x) {return((x - min(x)) / (max(x) - min(x)))}
normalize_dataset$age <- normalize(df$age)
normalize_dataset$overall <- normalize(df$overall)
head(normalize_dataset[, c("short_name", "age", "overall")])
```

Output:

```
> normalize_dataset <- df
> normalize <- function(x) {return((x - min(x)) / (max(x) - min(x)))}
> normalize_dataset$age <- normalize(df$age)
> normalize_dataset$overall <- normalize(df$overall)
> head(normalize_dataset[, c("short_name", "age", "overall")])
     short_name       age   overall
1 J. Bellingham 0.2142857 0.9772727
2   F. Valverde 0.3571429 0.9545455
3    J. Kimmich 0.5000000 0.9545455
4     A. Hakimi 0.3571429 0.9545455
5    N. Barella 0.4285714 0.9090909
6 H. Çalhanoğlu 0.5357143 0.8863636
>
```

The min-max normalization function was applied to the age and overall columns, transforming their values to a range between 0 and 1. For example, J. Bellingham's age was normalized to 0.21 (indicating he is relatively young) and his overall rating to 0.98 (indicating he is among the highest-rated players). J. Kimmich's age normalized to 0.50 (mid-range) and overall, to 0.95. The normalized dataset was stored separately in normalize_dataset to preserve the original data for further analysis.

Exploratory Data Analysis:

1. **Summary Statistics:**

Code:

summary(df)

Output:

```
> summary(df)
  short_name         player_positions       overall         potential        value_eur            wage_eur
 Length:18405       Length:18405       Min.   :47.00   Min.   :49.00   Min.   :        0   Min.   :     0
 Class :character   Class :character   1st Qu.:61.00   1st Qu.:67.00   1st Qu.:   475000   1st Qu.:  1000
 Mode  :character   Mode  :character   Median :66.00   Median :71.00   Median :  1000000   Median :  4000
                                       Mean   :65.77   Mean   :71.16   Mean   :  2931665   Mean   : 10095
                                       3rd Qu.:70.00   3rd Qu.:75.00   3rd Qu.:  2100000   3rd Qu.: 10000
                                       Max.   :91.00   Max.   :95.00   Max.   :174500000   Max.   :610000
      age          height_cm     weight_kg      preferred_foot          pace          shooting
 Min.   :16.00   Min.   :155   Min.   : 47.00   Length:18405       Min.   :30.00   Min.   :21.00
 1st Qu.:21.00   1st Qu.:177   1st Qu.: 70.00   Class :character   1st Qu.:63.00   1st Qu.:45.00
 Median :25.00   Median :182   Median : 75.00   Mode  :character   Median :69.00   Median :55.00
 Mean   :25.22   Mean   :182   Mean   : 75.14                      Mean   :68.44   Mean   :53.06
 3rd Qu.:29.00   3rd Qu.:187   3rd Qu.: 80.00                      3rd Qu.:75.00   3rd Qu.:62.00
 Max.   :44.00   Max.   :210   Max.   :105.00                      Max.   :97.00   Max.   :92.00
    passing        dribbling       defending         physic
 Min.   :25.00   Min.   :22.00   Min.   :15.00   Min.   :32.0
 1st Qu.:52.00   1st Qu.:58.00   1st Qu.:41.00   1st Qu.:59.0
 Median :58.00   Median :64.00   Median :57.00   Median :66.0
 Mean   :57.66   Mean   :63.01   Mean   :52.52   Mean   :64.9
 3rd Qu.:63.00   3rd Qu.:68.00   3rd Qu.:63.00   3rd Qu.:71.0
 Max.   :92.00   Max.   :93.00   Max.   :90.00   Max.   :91.0
>
```

The summary statistics provide an overview of all variable distributions in the cleaned dataset. The overall rating ranges from 47 to 91 with a mean of 65.77, indicating most players are mid-range rated. The potential rating ranges from 49 to 95 with a mean of 71.16, showing players generally have higher potential than their current rating. Player ages range from 16 to 44 years with a median of 25. Market values show significant variation, ranging from 0 to 174.5 million EUR with a mean of approximately 2.93 million EUR, indicating the distribution is heavily right-skewed with a few very high-value players. Weekly wages follow a similar skewed pattern, ranging from 0 to 610,000 EUR. Physical attributes like height (mean 182 cm) and weight (mean 75.14 kg) fall within expected ranges for professional athletes. Skill ratings (pace, shooting, passing, dribbling, defending, physic) all range approximately from 15-97, with means between 52 and 68.

More detailed statistics:

Code:

summary(df$age)
summary(df$overall)

table(df$preferred_foot)

Output:

```
> summary(df$age)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  16.00   21.00   25.00   25.22   29.00   44.00 
> summary(df$overall)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  47.00   61.00   66.00   65.77   70.00   91.00 
>
> table(df$preferred_foot)

 Left Right 
 4498 13907 
>
```

The age of players ranges from 16 to 44 years with a median of 25, indicating the dataset is slightly skewed towards younger players. The overall rating ranges from 47 to 91 with a median of 66 and mean of 65.77, showing most players are clustered in the mid-60s range. The preferred_foot distribution shows a clear dominance of right-footed players (13,907) compared to left-footed players (4,498), which reflects real-world football statistics where approximately 75% of players are right-footed.

Feature Analysis: Detailed Overall Rating Statistics

Code:

```
cat("Mean Overall:", mean(df$overall), "\n")
cat("Median Overall:", median(df$overall), "\n")
cat("SD Overall:", sd(df$overall), "\n")
cat("Min Overall:", min(df$overall), "\n")
cat("Max Overall:", max(df$overall), "\n")
```

Output:

```
> cat("Mean Overall:", mean(df$overall), "\n")
Mean Overall: 65.76652
> cat("Median Overall:", median(df$overall), "\n")
Median Overall: 66
> cat("SD Overall:", sd(df$overall), "\n")
SD Overall: 6.980364
> cat("Min Overall:", min(df$overall), "\n")
Min Overall: 47
> cat("Max Overall:", max(df$overall), "\n")
Max Overall: 91
```

The detailed statistics for the Overall rating reveal that the mean (65.77) and median (66) are very close, suggesting a nearly symmetric distribution. The standard deviation of 6.98 indicates moderate spread around the mean. The minimum rating is 47 and maximum is 91, giving a range of 44 points. Most players fall within one standard deviation of the mean (approximately 59 to 73), which represents the bulk of average-to-good professional players. The top-rated players (85+) represent elite footballers, while the lower end (below 55) represents reserve or youth squad players.

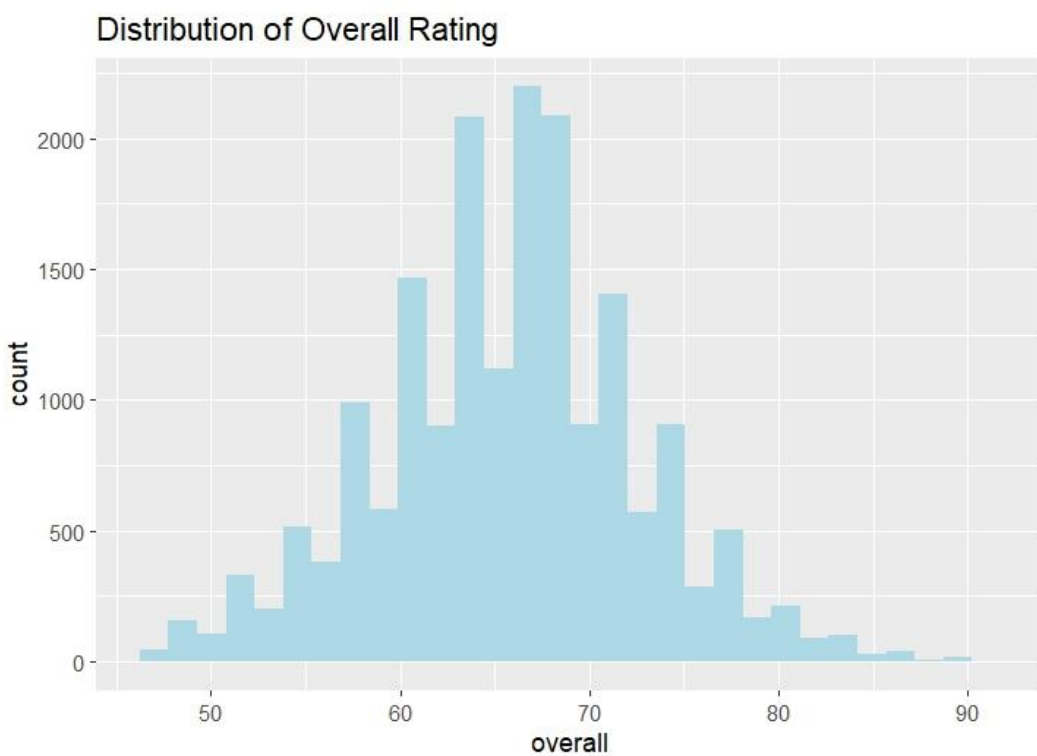
Visualizing Distributions:

1. **Histogram:**

Code:

```
ggplot(df, aes(overall)) +
  geom_histogram(bins = 30, fill = "lightblue") +
  ggtitle("Distribution of Overall Rating")
```

Output:



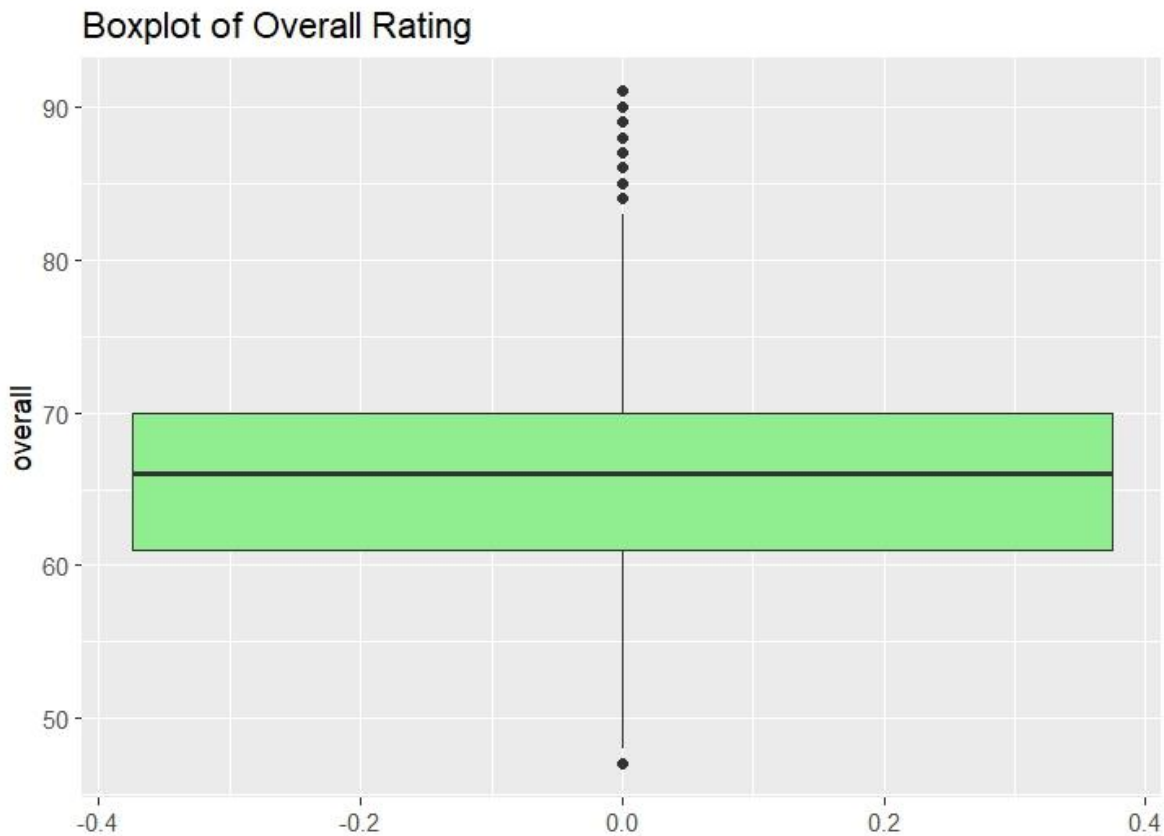
The histogram shows the frequency distribution of the Overall rating. The distribution is approximately bell-shaped with a slight left skew, peaking around 65-68 where over 2000 players are concentrated. The majority of players fall between 58 and 75, representing the mid-tier professional players. The left tail (below 55) contains fewer players representing lower-rated youth or reserve players, while the right tail (above 80) shows very few elite-level players, confirming that top-rated players are rare in the dataset.

2. **Boxplot:**

Code:

```
ggplot(df, aes(y = overall)) +
  geom_boxplot(fill = "lightgreen") +
  ggtitle("Boxplot of Overall Rating")
```

Output:



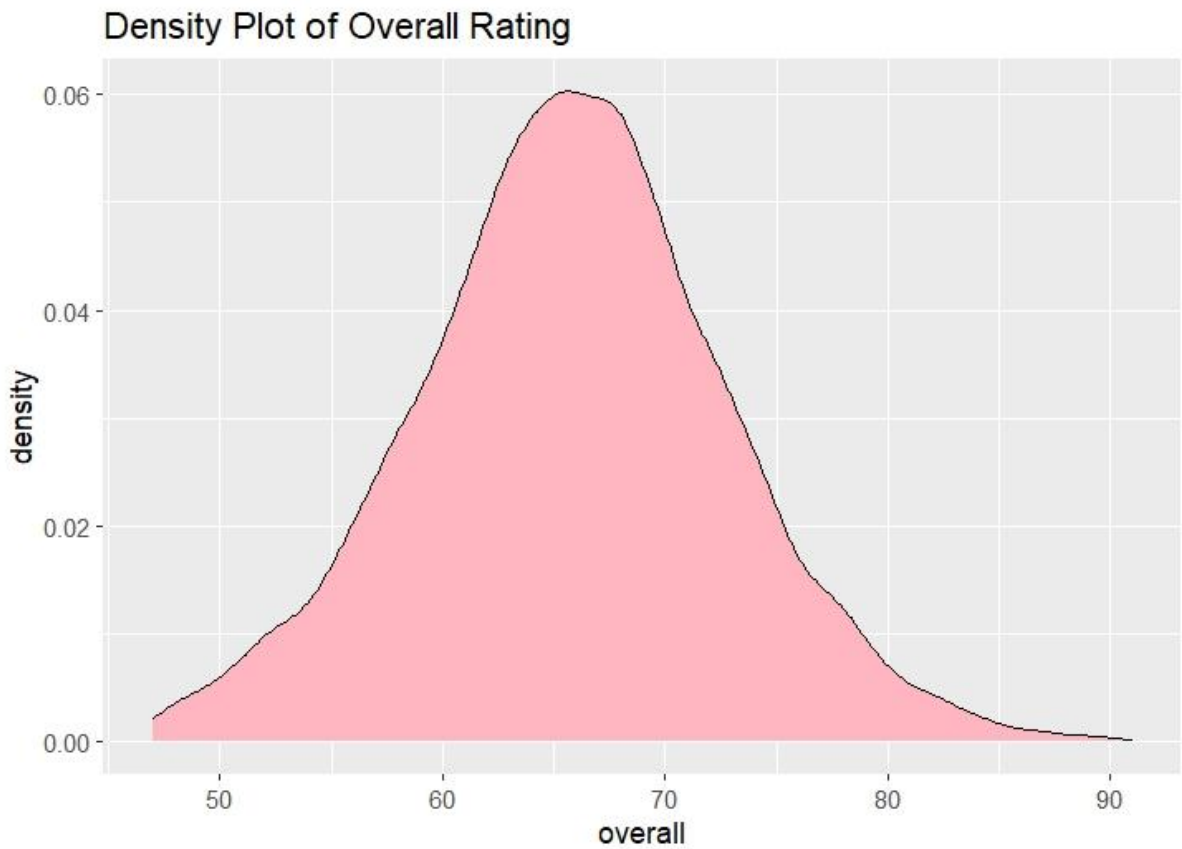
The boxplot displays the spread and distribution of the Overall rating. The median line sits at approximately 66, with the interquartile range (IQR) spanning from 61 (Q1) to 70 (Q3), meaning 50% of all players have ratings between 61 and 70. The lower whisker extends to around 48 and the upper whisker to about 82. The dots above the upper whisker represent outliers — these are elite-level players with ratings above 83 (such as Mbappé, Haaland, Bellingham). The single dot at the bottom represents the lowest-rated player at 47. No extreme artificial outliers remain, confirming the data is clean.

3. **Density Plot:**

Code:

```
ggplot(df, aes(overall)) +
  geom_density(fill = "lightpink") +
  ggtitle("Density Plot of Overall Rating")
```

Output:



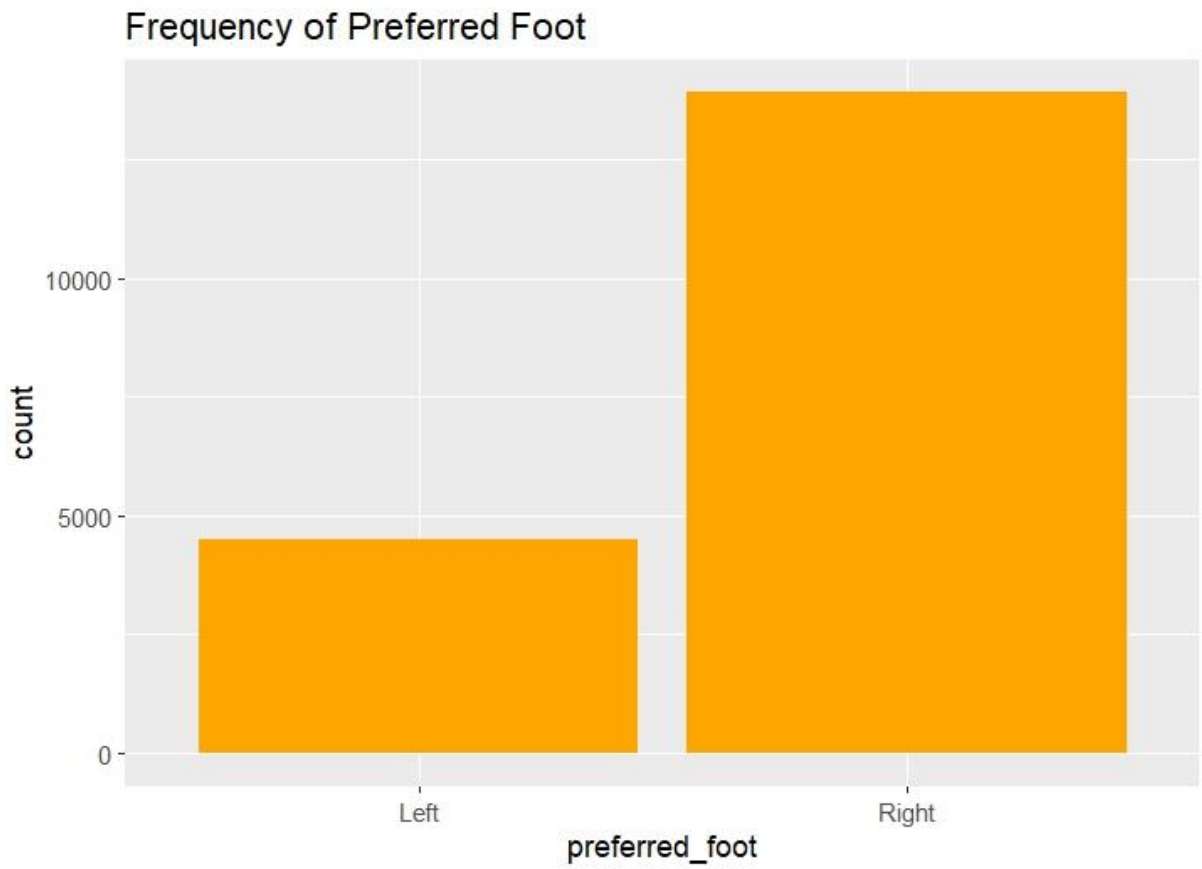
The density plot shows a smoothed probability distribution of the Overall rating. The curve reveals a unimodal (single peak) distribution with the peak around 66-68, confirming that most players are concentrated in this range. The distribution is slightly right-skewed with a longer tail extending towards higher ratings (80-91), indicating that elite players are rare. The left tail drops off more steeply below 50, showing very few extremely low-rated players. The smooth curve confirms an approximately normal distribution with a slight positive skew.

Category Frequency Plot:

Code:

```
ggplot(df, aes(preferred_foot)) +
  geom_bar(fill = "orange") +
  ggtitle("Frequency of Preferred Foot")
```

Output:



The bar chart displays the frequency distribution of the preferred_foot variable. Right-footed players dominate the dataset with approximately 13,907 players, while left-footed players account for 4,498. This roughly 75%-25% split accurately reflects real-world football demographics, where the majority of professional players are naturally right-footed. The chart also confirms that the two invalid categories ("Ambidextrous" and "none") introduced earlier were successfully cleaned, as only "Left" and "Right" remain.

Group Comparison:

Code:

```
tapply(df$overall, df$preferred_foot, mean)
tapply(df$overall, df$preferred_foot, sd)
```

Output:

```
> tapply(df$overall, df$preferred_foot, mean)
    Left    Right
66.45998 65.54223
> tapply(df$overall, df$preferred_foot, sd)
    Left    Right
6.719338 7.048372
>
```

The group comparison reveals that left-footed players have a slightly higher mean overall rating (66.46) compared to right-footed players (65.54), a difference of approximately 0.92 points. The standard deviation is slightly lower for left-footed players (6.72) compared to right-footed players (7.05), indicating that left-footed player ratings are marginally more consistent. This small difference could suggest that left-footed players who make it to professional level tend to be slightly higher rated on average, though the difference is not large enough to draw a strong conclusion.

Descriptive Statistics Across Target Classes (rating_class):

Code:

tapply(df$wage_eur, df$rating_class, mean)
tapply(df$wage_eur, df$rating_class, sd)
tapply(dfage, dfrating_class, mean)

Output:

```
> tapply(df$wage_eur, df$rating_class, mean)
Below Average       Average          Good
     1791.124      9232.814     43260.328
        Elite
   150452.632
> tapply(df$wage_eur, df$rating_class, sd)
Below Average       Average          Good
     1822.533      8544.536     30858.188
        Elite
    87341.561
> tapply(df$age, df$rating_class, mean)
Below Average       Average          Good
     23.01264      26.67311      27.54527
        Elite
     27.89474
>
```

The analysis shows a strong relationship between rating class and wages. Elite players earn an average of 150,453 EUR/week — 84 times more than Below Average players (1,791 EUR). Wage variation also increases with rating class, with Elite players showing the highest standard deviation (87,342 EUR). The mean age increases gradually from 23 (Below Average) to 27.89 (Elite), suggesting higher-rated players tend to be older due to the experience needed to reach peak performance.

Data Filtering::

Code:

```
data_filtered <- df %>% filter(age > 30)
cat("Players above age 30:", nrow(data_filtered), "\n")
head(data_filtered)
```

Output:

```
> data_filtered <- df %>% filter(age > 30)
> cat("Players above age 30:", nrow(data_filtered), "\n")
Players above age 30: 2823
> head(data_filtered)
      short_name player_positions overall
6  H. Çalhanoğlu          CDM, CM      86
10  A. Griezmann      ST, LM, CAM      85
17    R. De Paul     CM, CDM, CAM      84
22      M. Salah           RM, RW      91
23   M. Sabitzer          CDM, CM      80
25  João Cancelo       RB, LB, RM      84
   potential value_eur wage_eur age height_cm
6         86  47500000    61000  31       178
10        85  26000000   115000  34       176
17        84  34500000   100000  31       180
22        91  82000000   370000  33       175
23        80  15000000    52000  31       178
25        84  30500000    68000  31       182
   weight_kg preferred_foot pace shooting passing
6         69          Right   71       81      87
10        67           Left   77       86      85
17        70          Right   75       78      83
22        72           Left   89       88      86
23        76          Right   74       80      80
25        74          Right   83       73      84
   dribbling defending physic
6         82        81     73
10        88        59     73
17        83        76     83
22        90        45     76
23        80        76     76
25        84        78     74
>
```

The dataset was filtered to include only players above 30 years of age, resulting in 2,823 players out of 18,405 total (approximately 15.3%). The filtered output shows experienced veteran players like M. Salah (age 33, overall 91), A. Griezmann (age 34, overall 85), and H. Çalhanoğlu (age 31, overall 86). These senior players generally maintain high overall ratings, indicating that experienced professionals can sustain elite performance levels despite advancing age.

Balancing the Dataset:

Creating a categorical target variable based on overall rating:

Code:

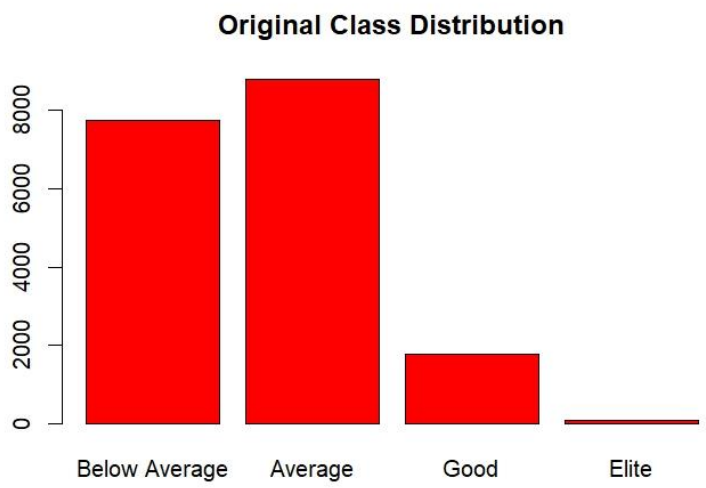
```
df$rating_class <- cut(df$overall,
                       breaks = c(0, 64, 74, 84, 100),
                       labels = c("Below Average", "Average", "Good", "Elite"))
idata <- df
```

Check original class distribution:

Code:

```
class_dist <- table(idata$rating_class)
class_dist
barplot(table(idata$rating_class), main = "Original Class Distribution", col = "red")
```

Output:



The bar chart and table reveal a highly imbalanced class distribution. The "Average" class (overall 65-74) dominates with 8,792 players, followed by "Below Average" (overall 0-64) with 7,751 players. The "Good" class (overall 75-84) has only 1,767 players, and the "Elite" class (overall 85-100) has just 95 players. This severe imbalance, where the largest class is nearly 93 times bigger than the smallest, could lead to biases in predictive modeling. Balancing techniques will be applied to address this issue..

Balancing Using Over Sampling::

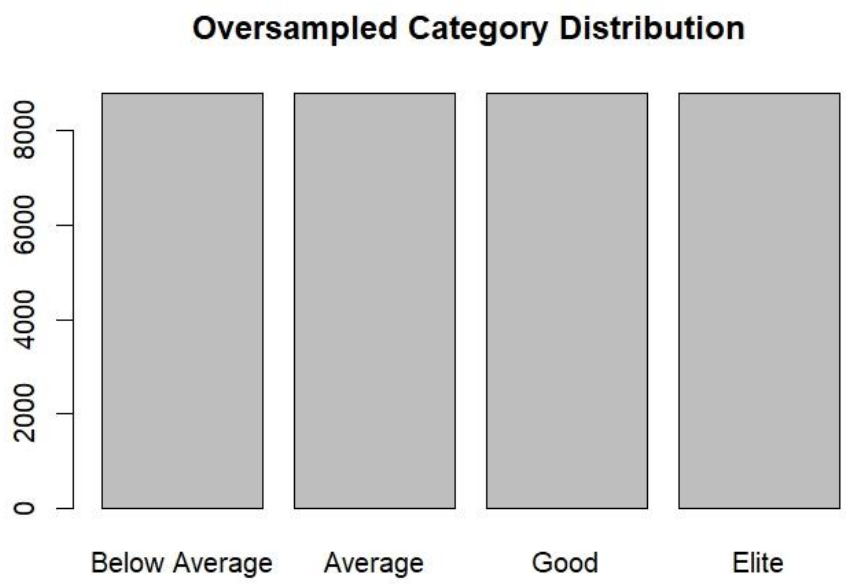
Duplicates minority-class samples to match the majority class size, creating a balanced dataset.

Code:

```
class_counts <- table(idata$rating_class)
max_count <- max(class_counts)
set.seed(100)
os_list <- lapply(names(class_counts), function(cl) {
  df <- filter(idata, rating_class == cl)
  if(nrow(df) < max_count) {
    df <- df %>% sample_n(max_count, replace = TRUE)
  }
  df
})

os_balanced_data <- bind_rows(os_list)
table(os_balanced_data$rating_class)
barplot(table(os_balanced_data$rating_class),
        main = "Oversampled Category Distribution")
```

Output:



```
> os_balanced_data <- bind_rows(os_list)
> table(os_balanced_data$rating_class)

Below Average       Average          Good
         8792          8792          8792
        Elite
         8792
> barplot(table(os_balanced_data$rating_class),
+         main = "Oversampled Category Distribution")
>
```

The oversampling technique successfully balanced the dataset by duplicating minority-class samples to match the majority class size of 8,792. All four classes (Below Average, Average, Good, and Elite) now contain equal numbers of 8,792 players each. The "Elite" class, which originally had only 95 players, was oversampled to 8,792 by randomly duplicating existing elite player records with replacement. The barplot confirms that all classes are now evenly distributed, eliminating the class imbalance that could cause bias in predictive modeling.

Splitting the Dataset:

Code:

```
n <- nrow(df)
random_index <- sample(1:n, size = 0.8*n)
train_data <- df[random_index, ]
test_data <- df[-random_index, ]
cat("Training set rows:", nrow(train_data), "\n")
cat("Testing set rows:", nrow(test_data), "\n")
```

Output:

```
> n <- nrow(df)
> random_index <- sample(1:n, size = 0.8*n)
> train_data <- df[random_index, ]
> test_data <- df[-random_index, ]
> cat("Training set rows:", nrow(train_data), "\n")
Training set rows: 14724
> cat("Testing set rows:", nrow(test_data), "\n")
Testing set rows: 3681
>
```

The dataset was split into training and testing sets using an 80/20 ratio. The training set contains 14,724 rows (80%) and the testing set contains 3,681 rows (20%). The rows were randomly selected using the sample() function to ensure an unbiased split. The training set will be used for model building and learning patterns, while the testing set will be reserved for evaluating model performance. This step ensures that the data is divided properly for model training and evaluation.

Conclusion:

In this project, a complete data-science workflow has been implemented on the FC26 FIFA Player Dataset. The raw data was subjected to systematic cleaning in terms of missing values, noisy entries, outliers, duplicated entries, and inconsistent formats. These were indeed crucial preprocessing steps towards making the dataset reliable, consistent, and analytically valid. Following cleaning, the now-refined dataset was subsequently employed for exploratory data analysis, whereby at each instance, descriptive statistics and visualizations provided meaningful insights into the distribution and behavior of key player attributes such as overall rating, potential, age, and wages. Subsequent steps include normalization, filtering, feature exploration, class balancing, and dataset splitting to further prepare data for downstream machine-learning tasks.

In general, the project shows that the structured data-science pipeline matters from raw data to a clean and interpretable form, to analyzing underlying trends, all the way to preparing the data for predictive modeling. The cleaned dataset is now well-organized, sound statistically, and ready for model development to contribute toward understanding and predicting player performance and ratings.